Introduction to computer programming (2023/2024)

Paolo Franciosa (credits to Massimo Lauria)

Exam February 12, 2024

Working methods: The exam text, the test files of the exercises and a synoptic table on Python syntax can be found in the `exam` folder on your desk, in the "Statistica-Laboratorio" virtual machine. The solutions to the exercises must be placed in the same folder `exam`. **First compile the file** `studente.txt` with the first name, surname and matriculation information

```
nome: Giacomo
cognome: Leopardi
matricola: 123456789
```

Not completing the `student.txt` file is like not handing in the assignment. For the exam, you can only have

- your ID or passport;
- beverages and snacks;

and you must leave your bags and backpacks in the corner of the room, with mobile phones and electronic devices switched off inside.

The grade is the sum of the points of the exercises, so it is possible to get 30 even without completing them all.

Use of test files: tests check very briefly the setting of the solutions and passing the tests does not at all imply that the solution is correct. Read the text of the exercises before starting to solve them. To complete each exercise exercise you must

- write a solution file **with the specified name**;
- have the function **with the specified name** inside;
- place the solution file in the `exam` folder;

- run the `python3 <testfile>` command in that folder

where `<testfile>` should of course be replaced with the name of the appropriate test file for the exercise you are working on.

There is an independent test file for each exercise, so you can work on the exercises one at a time with more ease. The result of each test is a screen (or several screens) showing

- whether the file with the correct name was found,
- whether the file contains the function with the correct name,
- whether calls to the function with different parameter values end by returning correct results.

For each function written, calls are made with different values of the parameters. The result of the tests is returned with the character

- `E` if the call cannot be executed,
- `F` if the function does not return the correct result,
- `.` if the function returns the correct result.

1 Largest value (1 points)

Write a function `largest_value(v_1, v_2)` that:

- takes two numbers `v_1` `v_2` as parameters;
- returns the maximum among the absolute values of `v_1` and `v_2`.

```
print(largest_value(-7, 5))
print(largest_value(4, 8))
print(largest_value(-3, -5.7))
```

```
7
8
5.7
```

Python function must be saved in file: `largest_value.py`

Test file: `test_largest_value.py`

2 Count even values (1 points)

Write a function `count_even(L)` that,

- takes a list `L` as parameter, in which each element is an `int` value;
- returns the number of even values in `L`. You can assume that `L` is a list of `int` numbers.

```
a = [2, -1, 0, 4, 8]
print(count_even(a))
```

```
4
```

Python function must be saved in file: `count_even.py`

Test file: `test_count_even.py`

3 Difference of lists (1 points)

Write a function `list_difference(a, b)` that:

- takes two lists `a`, `b`;
- returns a new list containing all item in `a` that are not in `b`.

You can **assume** that `a` has no duplicates and `b` has no duplicates.

Elements in the result must appear in the same order as in `a`.

```
x = [7, 5, 0, -1]
y = [-1, 6, 5]
print(list_difference(x, y))
```

```
[7, 0]
```

Python function must be saved in file: `list_difference.py`

Test file: `test_list_difference.py`

4 Create reduced dictionary (1 points)

Write a function `dict_reduced(d, threshold)` that:

- takes as arguments a dictionary `d` with values of type `int` or `float` and a number `threshold`;
- returns a new dictionary containing all the keys and associated values in `d` where the value is strictly greater than `threshold`.

You may assume that all values are numeric.

For example:

```
d = {"Mario": 1000, "Gianna" : 8000, "Alberto": 4000, "John": 750}

print(dict_reduced(d, 1000))
```

```
{'Gianna': 8000, 'Alberto': 4000}
```

Python function must be saved in file: `dict_reduced.py`

Test file: `test_dict_reduced.py`

5 Positions of dominant rows in a matrix (1 points)

Write a function `mat_dominant_rows(m)` that:

- takes as argument a matrix `m` of numbers, represented as a list of lists;
- returns a list containing the positions of all rows of `m` that dominate at least one row in `m`.

Given two lists `a` and `b` with same length, we say `a` dominates `b` if and only if each element of `a` is strictly larger than the corresponding element of `b`. For example, [1, 3, -4, 6] dominates [0, 1, -9, 0] because $1 > 0$, $3 > 1$, $-4 > -9$ and $6 > 0$.

We represent an $r \times c$ matrix in Python as a list with length r, where each element is a list of length c. You may assume that the argument is well formed, with $r \geq 1$, $c \geq 1$, and all the lists representing rows contain `c` numeric elements.

In the following example:

- row in position 1 dominates row in position 4,
- row in position 2 dominates both row in position 0, row in position 3 and row in position 4

thus the result is `[1, 2]`

```
m = [[0, 0, 0], [0, 1, 0], [2, 1, 1], [0, 0, 0], [-1, 0, -1]]
print(mat_dominant_rows(m))
```

```
[1, 2]
```

Python function must be saved in file: `mat_dominant_rows.py`

Test file: `test_mat_dominant_rows.py`

6 Cumulated list (1 points)

Write a function `cumulated_list(values)` that:

- expects a non-empty list `values` of numeric values as an argument;
- creates and returns a list having the same length as `values`, in which `i`-th element contains the sum of elements of `values` from `0`-th to `i`-th (included).

```
a = [3, 1, 4, 1, 5]
print(cumulated_list(a))
```

```
[3, 4, 8, 9, 14]
```

Python function must be saved in file: `cumulated_list.py`

Test file: `test_cumulated_list.py`